ASSIGNMENT 2

MACHINE LEARNING

Name: Aswani T.A

Roll No.: 14

CSE-A / S-7

1. Problem Statement

Forensic glass fragments recovered from crime scenes can provide useful evidence for identifying the type of glass involved in an incident. The objective of this assignment is to develop a machine learning model capable of classifying forensic glass fragments into different glass types using their refractive index and elemental composition.

The classification problem uses the refractive index and eight elemental composition features as input variables. The target variable represents six different types of glass.

A suitable machine learning algorithm is selected, trained and tested using the Glass Identification dataset. The performance of the model is evaluated using suitable classification metrics and visualized using graphs and plots.

The selected application is:

Forensic Glass Type Classification Using Random Forest

2. Dataset Description

The dataset used for this assignment is the UCI Glass Identification dataset, also available through the MASS package as the `fgl` dataset.

The original dataset contains 214 glass samples with nine numerical input features and one target class. After checking for duplicate observations, one duplicate was removed, resulting in 213 samples.

The input features are:

- Refractive Index (RI)
- Sodium (Na)
- Magnesium (Mg)
- Aluminium (Al)
- Silicon (Si)
- Potassium (K)
- Calcium (Ca)
- Barium (Ba)

- Iron (Fe)

The target variable contains six glass categories:

- WinF – Window glass, float processed
- WinNF – Window glass, non-float processed
- Veh – Vehicle window glass
- Con – Container glass
- Tabl – Tableware glass
- Head – Headlamp glass

The refractive index is represented on the shifted scale:

$$RI_{scaled} = (RI - 1.518) \times 1000$$

3. Dataset Distribution

The dataset is imbalanced because different glass categories contain different numbers of samples.

Table 1: Distribution of Glass Classes

Glass Type	**Samples**	**Percentage**
WinNF	76	35.7%
WinF	69	32.4%
Head	29	13.6%
Veh	17	8.0%
Con	13	6.1%
Tabl	9	4.2%

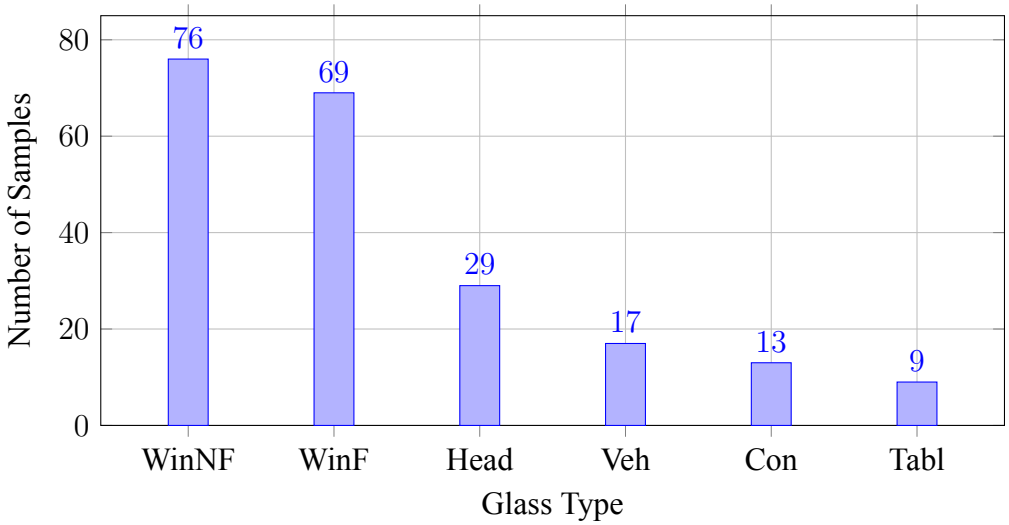


Figure 1: Class Distribution of the Glass Dataset

4. Data Preprocessing

The following preprocessing steps were performed:

1. Removed the row index because it is not a predictive feature.
2. Checked for missing values.
3. Checked for duplicate observations.
4. Removed one duplicate observation.
5. Selected the nine input features.
6. Encoded the target class using LabelEncoder.
7. Divided the dataset into training and testing sets.
8. Used a stratified 75:25 train-test split.
9. Applied StandardScaler inside a Pipeline.
10. Used class weighting to handle class imbalance.

The final dataset contained:

$$213 samples$$

The training and testing sets contained:

$$159 training samples$$

and

$$54 testing samples$$

respectively.

4.1. Main Preprocessing Code

The following is the main preprocessing portion of the Python implementation.

```
df = pd.read_csv(CSV)

df = df.drop(columns=["rownames"])

log("Missing values:",
    int(df.isna().sum().sum()))

log("Duplicate rows:",
    int(df.duplicated().sum()))

df = df.drop_duplicates().reset_index(drop=True)

features = [
    "RI", "Na", "Mg", "Al",
    "Si", "K", "Ca", "Ba", "Fe"
]

le = LabelEncoder()

y = le.fit_transform(df["type"])

X = df[features]
```

5. Exploratory Data Analysis

Exploratory data analysis was performed to understand the distribution of the data and the relationships between the input features.

The correlation analysis showed a strong positive relationship between refractive index and calcium.

$$r(RI, Ca) \approx 0.81$$

Magnesium showed negative relationships with Aluminium, Barium and Calcium.

Feature distributions also showed that Magnesium and Aluminium can provide useful separation between several glass categories.

5.1. Feature Correlation

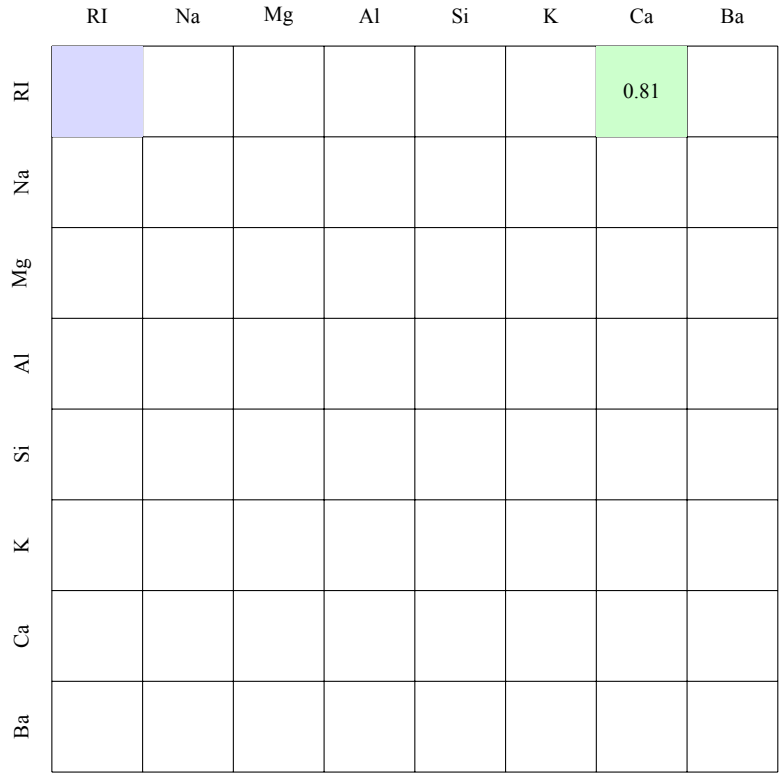


Figure 2: Illustration of Important Feature Correlation

6. Machine Learning Algorithm Selection

Five machine learning algorithms were compared using five-fold stratified cross-validation.

The algorithms were:

1. Logistic Regression

2. K-Nearest Neighbours
3. Support Vector Machine with RBF kernel
4. Gradient Boosting
5. Random Forest

Macro-F1 was selected as the main comparison metric because the dataset is imbalanced.

6.1. Main Model Training Code

The main model comparison code used in the implementation is:

```
models = {

    "Logistic Regression":
        Pipeline([
            ("sc", StandardScaler()),
            ("m", LogisticRegression(
                max_iter=2000,
                class_weight="balanced"
            ))
        ]),

    "KNN":
        Pipeline([
            ("sc", StandardScaler()),
            ("m", KNeighborsClassifier(
                n_neighbors=5
            ))
        ]),

    "SVM (RBF)":
        Pipeline([
            ("sc", StandardScaler()),
            ("m", SVC(
                class_weight="balanced",
                random_state=SEED
            ))
        ]),

    "Gradient Boosting":
        Pipeline([
            ("sc", StandardScaler()),
            ("m", GradientBoostingClassifier(
                random_state=SEED
            ))
        ]),
```

```
    "Random Forest":
        Pipeline([
            ("sc", StandardScaler()),
            ("m", RandomForestClassifier(
                n_estimators=300,
                class_weight="balanced",
                random_state=SEED
            ))
        ])
}
```

6.2. Cross-Validation

A five-fold StratifiedKFold cross-validation strategy was used.

```
cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=SEED
)

for name, pipe in models.items():

    s = cross_val_score(
        pipe,
        X_tr,
        y_tr,
        cv=cv,
        scoring="f1_macro",
        n_jobs=-1
    )

    cv_res[name] = s
```

7. Model Comparison

The cross-validation results are shown below.

Table 2: Five-Fold Cross-Validation Results

Model	Mean Macro-F1	Standard Deviation
Random Forest	0.682	0.114
SVM RBF	0.650	0.077
Gradient Boosting	0.623	0.120
Logistic Regression	0.593	0.074
KNN	0.555	0.111

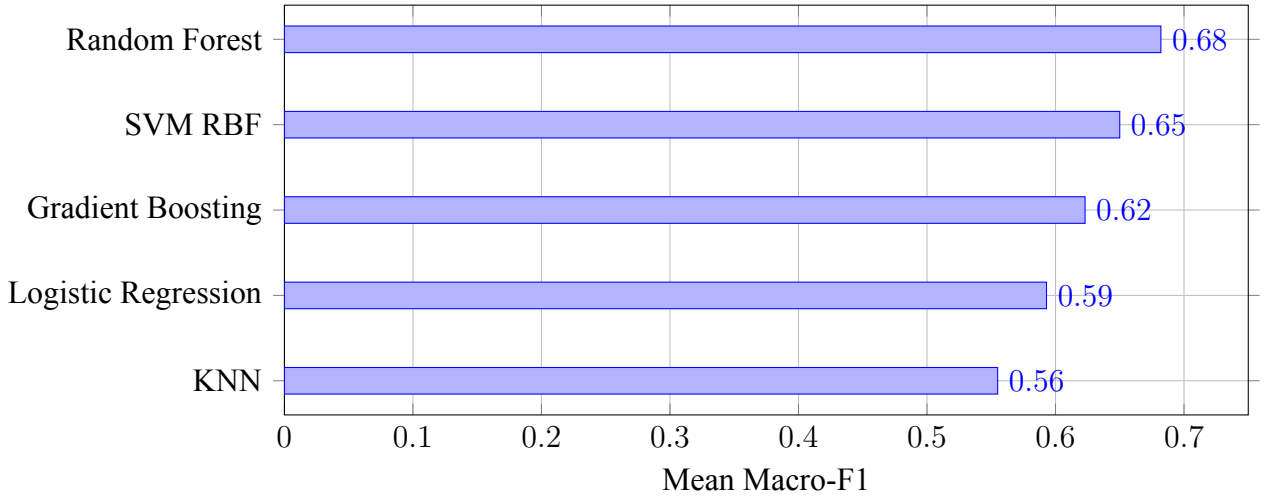


Figure 3: Comparison of Machine Learning Models

8. Random Forest Algorithm

Random Forest was selected because it is suitable for small tabular datasets and can model non-linear relationships between the input features and glass classes.

The advantages include:

- Ability to model non-linear decision boundaries.
- Ability to capture feature interactions.
- Robust performance on tabular datasets.
- Support for class weighting.
- Availability of feature importance values.

9. Hyperparameter Tuning

GridSearchCV was used to improve the Random Forest model.

The parameters searched were:

- Number of estimators: 200, 400
- Maximum depth: None, 8, 15
- Minimum samples per leaf: 1, 2
- Maximum features: sqrt, 0.5

The main tuning code was:

```
rf_pipe = Pipeline([
    ("sc", StandardScaler()),
    ("m", RandomForestClassifier(
        class_weight="balanced",
        random_state=SEED
    ))
])

grid = {
    "m__n_estimators": [200, 400],
    "m__max_depth": [None, 8, 15],
    "m__min_samples_leaf": [1, 2],
    "m__max_features": ["sqrt", 0.5]
}

gs = GridSearchCV(
    rf_pipe,
    grid,
    cv=cv,
    scoring="f1_macro",
    n_jobs=-1
)

gs.fit(X_tr, y_tr)

model = gs.best_estimator_
```

The best configuration was:

Table 3: Best Random Forest Configuration

Parameter	Value
Number of Trees	200
Maximum Depth	8
Minimum Samples per Leaf	2
Maximum Features	sqrt

The best cross-validation macro-F1 score after tuning was approximately:

$$0.717$$

10. Model Testing

The tuned Random Forest model was evaluated on the held-out test set.

The main prediction code was:

```
pred = model.predict(X_te)

acc = accuracy_score(y_te, pred)

precision = precision_score(
    y_te,
    pred,
    average="macro",
    zero_division=0
)

recall = recall_score(
    y_te,
    pred,
    average="macro",
    zero_division=0
)

f1 = f1_score(
    y_te,
    pred,
    average="macro",
    zero_division=0
)
```

11. Evaluation Results

The final tuned Random Forest model produced the following results:

Table 4: Final Test Set Performance

Evaluation Metric	Score
Accuracy	0.722
Precision (Macro)	0.709
Recall (Macro)	0.708
F1 Score (Macro)	0.709
F1 Score (Weighted)	0.723

The model correctly classified 39 out of 54 test samples.

The final accuracy was approximately:

$$72.2\%$$

and the macro-F1 score was:

$$0.709$$

12. Class-wise Results

Table 5: Class-wise Classification Performance

Class	Precision	Recall	F1	Support
Con	0.67	0.67	0.67	3
Head	0.88	0.88	0.88	8
Tabl	1.00	1.00	1.00	2
Veh	0.25	0.25	0.25	4
WinF	0.68	0.72	0.70	18
WinNF	0.78	0.74	0.76	19

12.1. Per-Class F1 Score

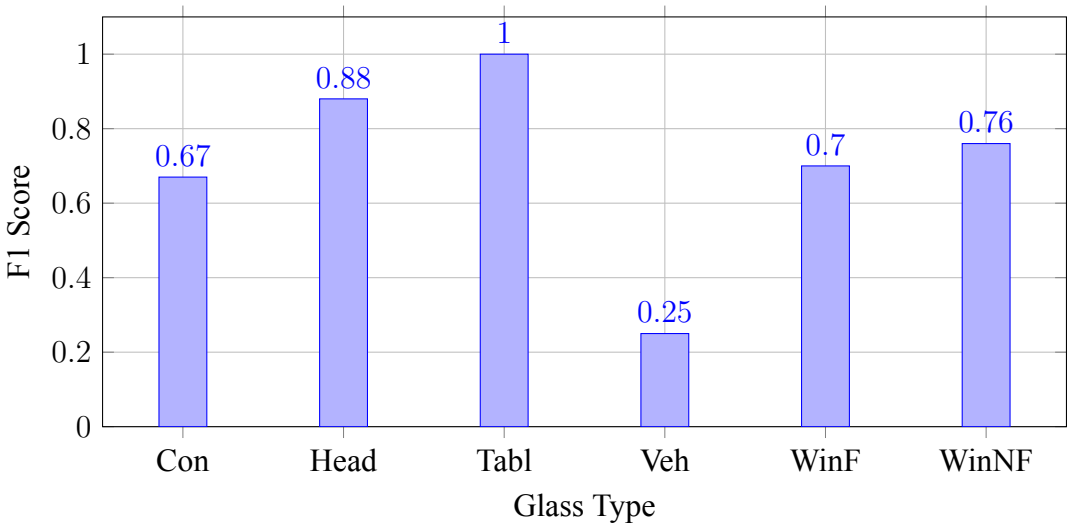


Figure 4: F1 Score for Each Glass Class

13. Confusion Matrix

The confusion matrix provides information about the prediction errors between the different glass categories.

The model performed relatively well for Headlamp and Tableware glass, while Vehicle glass was more difficult to classify.

The original Python implementation generates the confusion matrix using:

```
cm = confusion_matrix(y_te, pred)

ConfusionMatrixDisplay(
    cm,
    display_labels=le.classes_
).plot(
    cmap="Blues",
    colorbar=False
)
```

14. Feature Importance

Random Forest provides feature importance values that can be used to identify the variables that contribute most strongly to classification.

The implementation used:

```
imp = pd.Series(
    model.named_steps["m"].feature_importances_,
    index=features
).sort_values()

imp.plot.barh()

plt.title(
    "Random Forest feature importance"
)

plt.xlabel(
    "Mean decrease in impurity"
)
```

The analysis indicated that Magnesium, Sodium, Aluminium and Potassium were among the important variables, while Iron contributed relatively little.

15. Analysis of Results

The final Random Forest model achieved an accuracy of 72.2% and a macro-F1 score of 0.709.

The model performed considerably better than the approximate majority-class baseline of 36%.

The Headlamp class achieved an F1 score of 0.88, indicating strong classification performance.

The Tableware class achieved an F1 score of 1.00. However, this class contained only two test samples, so the perfect score should be interpreted carefully.

Vehicle glass was the weakest class, with an F1 score of 0.25. Some Vehicle samples were confused with Window glass.

The two window categories, WinF and WinNF, also showed considerable overlap. This explains some of the classification errors.

The cross-validation macro-F1 score of 0.717 and the test macro-F1 score of 0.709 are relatively close, suggesting that the model does not show a major indication of overfitting.

16. Limitations

The main limitations of the implementation are:

1. The dataset is relatively small.

2. The class distribution is imbalanced.
3. Some classes have very few samples.
4. Vehicle glass is difficult to distinguish from some window glass.
5. Test-set performance can vary because of the small number of samples.
6. A larger dataset could provide more reliable generalization.

17. Future Improvements

Possible improvements include:

- Collecting more samples for minority classes.
- Using repeated stratified cross-validation.
- Applying SMOTE or other resampling techniques.
- Creating additional chemical composition features.
- Testing ensemble approaches.
- Combining Random Forest with SVM.

18. Optional Streamlit Interface

As an optional extension, a simple Streamlit interface was developed.

The interface allows the user to enter the refractive index and chemical composition values of a glass fragment. The trained Random Forest model then predicts the glass type and displays the prediction confidence.

The application can be started using:

```
streamlit run app.py
```

18.1. Main Streamlit Code

The main portion of the Streamlit application is:

```
@st.cache_resource
def load():

    bundle = joblib.load(
        os.path.join(
            HERE,
            "glass_model.joblib"
        )
    )

    data = pd.read_csv(
        os.path.join(
            HERE,
            "glass.csv"
        )
    ).drop(
        columns=["rownames"]
    )

    return bundle, data


bundle, data = load()

model = bundle["model"]
le = bundle["label_encoder"]
features = bundle["features"]
```

The user interface is created using:

```
st.title(
    "Forensic Glass Type Classifier"
)

st.write(
    "Enter the refractive index and "
    "chemical composition of a glass "
    "fragment to predict its type."
)

cols = st.columns(3)

values = {}

for i, f in enumerate(features):

    lo = float(data[f].min())
    hi = float(data[f].max())
    med = float(data[f].median())

    values[f] = cols[i % 3].number_input(
```

```
        f,
        min_value=lo - 1,
        max_value=hi + 1,
        value=med
    )
```

The prediction is generated using:

```
if st.button("Predict", type="primary"):

    x = pd.DataFrame(
        [values]
    )[features]

    proba = model.predict_proba(x)[0]

    best = proba.argmax()

    code = le.classes_[best]

    st.success(
        f"Predicted type: "
        f"{FULL[code]} "
        f"(confidence "
        f"{proba[best]:.0%})"
    )

    st.bar_chart(
        pd.Series(
            proba,
            index=[
                FULL[c]
                for c in le.classes_
            ],
            name="Probability"
        )
    )
```

19. Software Requirements

The implementation requires Python and the following libraries:

- Python
- NumPy
- Pandas

- Scikit-learn
- Matplotlib
- Seaborn
- Joblib
- Streamlit

The main implementation files are:

```
glass_classifier.py
app.py
glass.csv
glass_model.joblib
results.txt
figures/
```

20. Conclusion

This assignment demonstrated the complete development of a machine learning application for forensic glass type classification.

The Glass Identification dataset was selected and analysed. Data preprocessing included duplicate checking, feature selection, label encoding and stratified train-test splitting.

Five machine learning algorithms were compared using five-fold stratified cross-validation. Random Forest produced the highest initial macro-F1 score and was selected for further tuning.

After hyperparameter tuning, the Random Forest model achieved an accuracy of 72.2% and a macro-F1 score of 0.709 on the held-out test set.

The model performed particularly well for Headlamp and Tableware glass, while Vehicle glass remained more challenging. The analysis demonstrates the importance of using class-wise evaluation metrics in an imbalanced classification problem.

An optional Streamlit interface was also developed to allow users to enter glass composition values and obtain a predicted glass type with confidence information.

Overall, the assignment satisfies the requirements of selecting a machine learning application, preprocessing the dataset, selecting and training a suitable algorithm, evaluating the model, displaying results graphically, and analysing the obtained results.

21. References

1. German, B. S., Glass Identification Database, UCI Machine Learning Repository.
2. Venables, W. N. and Ripley, B. D., *Modern Applied Statistics with S*, MASS package, `fgl` dataset.
3. Scikit-learn documentation for Random Forest, GridSearchCV, StandardScaler and classification metrics.
4. Streamlit documentation for developing interactive machine learning applications.